Projeto Pokedex CLI

Especificação do MVP

Objetivo

Criar uma aplicação de linha de comando em Python capaz de consultar informações de Pokémon através de uma API pública, permitindo pesquisar Pokémon, listar resultados e visualizar seus detalhes.

API

Utilizar a **PokéAPI** como fonte dos dados. As informações de Pokémon devem ser obtidas da API, sem cadastro manual dos Pokémon na aplicação.

```
Endpoint de referência: https://pokeapi.co/api/v2/pokemon/{name_or_id}
```

1. Funcionalidades obrigatórias

1.1 Buscar Pokémon

O usuário deve poder pesquisar um Pokémon informando seu nome ou número.

```
> pikachu
> 25
```

A aplicação deve apresentar, no mínimo: nome, número/ID, tipos, altura, peso e habilidades.

1.2 Listar Pokémon

Permitir listar Pokémon disponíveis na API utilizando paginação. Não é necessário carregar todos os Pokémon de uma vez.

```
Exemplo de interação:
> list

Showing Pokémon 1-20

1. Bulbasaur
2. Ivysaur
3. Venusaur
...

[n] Next page
[p] Previous page
[q] Quit
```

1.3 Detalhes do Pokémon

Ao consultar um Pokémon, apresentar também seus atributos básicos e stats.

Stats mínimos: HP, Attack, Defense, Sp. Atk, Sp. Def e Speed.

2. Menu da aplicação

A aplicação deve possuir um menu principal de linha de comando, por exemplo:

```
====================
POKEDEX
====================

1. Search Pokémon
2. List Pokémon
3. Exit

Choose an option:
```

3. Requisitos técnicos

- Utilizar Python.
- Consumir a PokéAPI através de HTTP.
- Utilizar uma biblioteca HTTP, como requests.
- Trabalhar com as respostas JSON da API.
- Não utilizar banco de dados neste MVP.
- Não utilizar LLM, frameworks de agentes ou qualquer recurso de IA neste MVP.

4. Orientação a objetos e organização

O projeto deve utilizar uma abordagem **orientada a objetos de forma moderada**. Não é necessário transformar tudo em classes. A ideia é usar POO onde ela ajuda a representar o domínio e separar responsabilidades.

Criar pelo menos uma representação própria de Pokémon, contendo os dados utilizados pela aplicação:

```
Pokemon
■■■ id
■■■ name
■■■ height
■■■ weight
■■■ types
■■■ abilities
■■■ stats
```

A API retorna muito mais dados do que a aplicação precisa. O objeto Pokemon deve conter somente os dados relevantes para o MVP.

5. Organização sugerida

Uma possível organização é:

```
pokedex/
■
■■■ main.py
■
■■■ models/
■ ■■■ pokemon.py
■
■■■ services/
■ ■■■ pokeapi.py
■
■■■ cli/
■ ■■■ menu.py
```

```
■
■■■ utils/
■■■ ...
```

A estrutura acima é uma sugestão, não uma obrigação. O importante é manter separadas as responsabilidades de interação com o usuário, comunicação com a API e representação dos dados.

6. Tratamento de erros

A aplicação não deve exibir uma stack trace para o usuário em situações esperadas. Deve apresentar mensagens claras, por exemplo:

```
Pokémon not found.
```

- Pokémon inexistente.
- Entrada inválida do usuário.
- Erro HTTP.
- API indisponível.
- Timeout.
- Resposta inesperada ou inválida.

7. Objetivos de aprendizado

O projeto deve servir como exercício prático dos seguintes conceitos:

- Funções e organização de código.
- Classes e orientação a objetos.
- Módulos e imports.
- Type hints, quando apropriado.
- Exceptions e tratamento de erros.
- JSON.
- HTTP e APIs REST.
- Transformação de dados externos em objetos Python.
- Separação de responsabilidades.

Regra do projeto: primeiro fazer o MVP funcionar de ponta a ponta. Refatorações e melhorias de arquitetura ficam para depois da primeira versão funcional.